Lesson notes · for you

The OWASP Top 10 for LLM applications

AI security · University · Lesson 2

Time
20 min

Materials
The ten risks (on the slides); An LLM app you are building, or a design on paper; A phone or computer for the escaping test, if you have one

Learning goals

- Name the ten OWASP risks for LLM apps, with an example of each.
- Treat model output as untrusted: escape it before it is used.
- Keep secrets and access rules out of prompts.

Activity: Ten-risk review

1. Sort (5 min): Write the ten risks on cards (number and name). With a partner, sort them into what goes in, what comes out and the system around the model, then check against the slide.
2. Review (8 min): For an app you are building, or a made-up design, write one line per risk: could it happen here, and what is the evidence?
3. Fix (5 min): Pick your three biggest risks and write a control for each that changes the design (escape output, remove a tool, add a limit), not just the prompt.
4. Test (2 min): For one control, write the test that would prove it works, such as escaping five tricky replies.

Review your own app or a made-up design. Never test a live app without written permission.

Discussion

- Why is the model's reply untrusted, even though your app asked for it?
- Which risk is easiest to fix by design?
- Why doesn't “never reveal this” protect a secret?

Check yourself

- Write one risk from the ten that applies to an app you use, and its control.

Try it this week
Take an LLM app you use or are building and go down the ten risks: one line each, could it happen here and what stops it. Then fix the three you can fix fastest.

Worksheet

The OWASP Top 10 for LLM applications

AI security · University · Lesson 2

Name _____ Class _____ Date _____

1 **A study assistant can delete files, but its job is only to read them. Which risk is that?**

◯ Misinformation ◯ Unbounded consumption ◯ Excessive agency

2 **Match each OWASP risk to an example of it.**

LLM01 Prompt injection	• • • •	It quotes another student's essay
LLM02 Sensitive information disclosure	• • • •	One user's endless requests run up a huge bill
LLM06 Excessive agency	• • • •	A web page it summarises tells it to ignore its rules
LLM10 Unbounded consumption	• • • •	It can delete files when it only needs to read them

3 **The course page shows the chatbot's replies. Finish escape_html(t) so the characters & < > " ' become & < > " '. Mind the order: the starter forgets the ampersand.**

```
def escape_html(t):
    t = t.replace("<", "&lt;")
    t = t.replace(">", "&gt;")
    return t

reply = 'Mark: <b>9</b> & "well done"'
print(escape_html(reply))
```

Write your program:

4 **What does this print?**

```
def bad(t):
    t = t.replace("<", "&lt;")
    return t.replace("&", "&amp;")

print(bad("<b>"))
```

◯ <b> ◯ <b> ◯ <b>

5 **Which of these keep a secret safe from system prompt leakage? Choose all that apply.**

☐ Tell the model “never reveal this” ☐ Decide who may see what in code, not in the prompt

☐ Keep API keys in the app's code or vault, never in the prompt ☐ Give the model only this user's data

☐ Write the secret in another language

✎ **Pick an LLM app you use. Which of the ten risks worries you most for it, and what would you check first?**

Check your answers

The OWASP Top 10 for LLM applications

AI security · University · Lesson 2

1 **Excessive agency**

Holding more tools or permissions than the job needs is excessive agency.

2 **LLM01 Prompt injection — A web page it summarises tells it to ignore its rules**
LLM02 Sensitive information disclosure — It quotes another student's essay
LLM06 Excessive agency — It can delete files when it only needs to read them
LLM10 Unbounded consumption — One user's endless requests run up a huge bill

Each risk has a typical shape: orders hidden in input, private data in output, powers the job doesn't need, and use with no limit.

3

```
def escape_html(t):
    t = t.replace("&", "&amp;")
    t = t.replace("<", "&lt;")
    t = t.replace(">", "&gt;")
    t = t.replace('"', "&quot;")
    return t.replace("'", "&#x27;")

reply = 'Mark: <b>9</b> & "well done"'
print(escape_html(reply))
```

Escaping turns markup in the reply into text the browser shows instead of runs.

4 **<b>**

< becomes <, then replacing & turns that new & into &, giving <b>.

5 **Keep API keys in the app's code or vault, never in the prompt; Decide who may see what in code, not in the prompt; Give the model only this user's data**

Assume the system prompt will be read. Secrets and access rules must live in code, and the model should only ever hold data this user may see.